

Resenha Crítica

1- Informações do Artigo

Título: Aplicação de desempenho e segurança de diferentes implementações do sistema de criptografia RSA

Autores e afiliação: Ana Carla Quallio Pora e Rodrigo Empisolo (DCC da UFPR); Daniel Macêdo Batista (DCC da USP)

Veículo de publicação: Anais eletrônicos do XXV SBSeg 2025, WT+ICG, publicado pela Sociedade Brasileira de Computação (SBC)

Ano de publicação: Publicado em 2025. Deriva de um trabalho de conclusão de curso, complementando resultados ~~parciais~~ preliminares dos mesmos autores no SBSeg 2024

2- Contexto e Objetivos

Motivação: O crescimento da troca de informações entre dispositivos tornou a criptografia essencial para garantir confidencialidade, integridade e autenticidade. O RSA, proposto em 1978, é um dos sistemas de chave pública mais usados, mas seu desempenho e segurança variam conforme a implementação, e a literatura ainda não compara sistematicamente essas variáveis entre implementações do mesmo algoritmo.

Problema: Os trabalhos relacionados costumam propor variações teóricas do RSA (ex: múltiplos primos, paralelização) ou compará-lo a outros algoritmos de chave pública, como ECC e Diffie-Hellman. Faltou, porém, uma investigação que explore como a escolha da linguagem de programação e da biblioteca criptográfica impacta tanto o tempo de execução quanto a ~~memória~~ exposição de dados sensíveis na memória do programa.

3- Desempenhamento de pesquisa

Metodologia: Estudo experimental de microbenchmarking em 3 etapas. Primeiro, selecionaram-se seis linguagens populares (C, C++, Java, JavaScript, Python e Rust) e, para cada uma, duas bibliotecas criptográficas conhecidas. Em seguida, mediu-se o tempo de geração de chaves, cifração e decifração usando ferramentas nativas de benchmarking, em uma única máquina, com chaves de 2048, 3072 e 1096 bits e uma mensagem teste de 30 bytes, repetindo as medições em conjunto de 100 a 10000 iterações e tomando a média de 5 execuções. Por fim, os autores compraram os padrões usados para armazenar chaves e realizaram uma análise de segurança prática com a ferramenta "gcore", gerando um despejo de memória de cada processo e buscando fragmentos das chaves privadas com o comando strings e scripts em python

Resultados:

- Geração de chaves: crypto++ (C++) e Cryptography (python) foram o mais rápido
- Cifração/decifração: Melhor C, Rust, Python (todas OpenSSL) → Pior: JavaScript
- Técnicas de segurança específicas foram identificadas, assim como vulnerabilidades pontuais
- Das bibliotecas de JavaScript foi possível reconstruir completamente a chave ~~de~~ através de fragmentos

Conclusão:

- Implementações baseadas em SSL apresentam melhor desempenho
- Cada biblioteca introduz mecanismos de segurança próprios que afetam desempenho e complexidade
- A contribuição central é a evidência prática de que a chave privada pode ser totalmente recuperada nas implementações JavaScript
- Autores reconhecem se tratar de um trabalho em andamento, derivado de um TCC, sugerindo como próximos passos testar outras configurações de Hardware, memória e tamanho de chave

4- Minha Análise Crítica

O artigo se destacou por mim por ir além da simples comparação de desempenho, mostrando uma análise de segurança prática e reproduzível que revelou uma vulnerabilidade concreta em uma das maiores linguagens de programação.

Por mais interessante e importante que tenha sido estes experimentos, os testes rodaram todos em uma única máquina, o que limita a generalização dos resultados para outros sistemas e ~~OS~~ SOs. Uma outra "problema" se deve ao fato de que o artigo não explora a causa original do problema.

Em conjunto, considero o artigo um bom artigo e um bom exemplo de pesquisa aplicada em segurança, com metodologia clara e com achado relevante e concreto.